SentinelRL: Budgeted Adaptive Cyber Defence in a Partially Observed Network

Team: [TEAM MEMBER(S)] | GitHub: [GITHUB URL]

Abstract. We study whether a Deep Q-Network (DQN) can make useful defensive decisions in a synthetic enterprise network with noisy alerts, stochastic lateral movement, and only six response actions per episode. The custom environment supports inspection, patching, isolation and decoy deployment. DQN improves over random and patch-priority policies, but not over the strongest hand-coded defenders in this experiment.

1. Problem and Contribution

Enterprise defenders must decide which assets to inspect, harden, isolate or use as decoys while incidents evolve and response capacity is limited. The action sequence matters: inspecting a host reveals its status, isolation prevents lateral spread, patching reduces future entry risk, and each intervention consumes one of only six response tokens. Alerts may be missed or false, so the agent acts on uncertain evidence rather than true compromise labels.

RL-based cyber defence and network simulators are established research areas, including CyberBattleSim, MininetGym and recent autonomous-defence reviews [1–3]. SentinelRL claims neither a new RL algorithm nor a new research domain. Its coursework contribution is a six-host POMDP combining a strict response budget, approximate host-belief features, four defensive actions and an attack-rate sensitivity test.

2. POMDP Formulation

The underlying state includes hidden compromise flags, host controls, telemetry, beliefs, budget and time. The DQN observes 58 values: nine per host (belief, alert, inspected/confirmed status, vulnerability, criticality, patched, isolated, decoy), plus four global features for budget, time, detections and newly compromised hosts. True compromise flags are hidden.

The 25 discrete actions are wait, inspect i, patch i, isolate i, or deploy a decoy on host i ($i = 0, \ldots, 5$). A mask excludes unavailable actions; wait is always valid. Each intervention consumes one of six tokens. Episodes last 48 steps. An attacker starts at the gateway or staff laptop and can spread across a fixed six-host graph.

For target host j, each compromised neighbour attempts lateral movement with probability

$$p_j = 0.23 v_j m_j d_j, \quad m_j = \begin{cases} 0.16, & \text{patched}, \\ 1, & \text{otherwise}, \end{cases}$$

Here v_j is vulnerability and $d_j = 0.10$ with a decoy, otherwise 1. Alert sensitivity is 0.55 and false-positive probability 0.08. All values are synthetic; the factorized belief update is approximate.

3. Reward and Learning

The reward sums event terms. Here c_i is host criticality; N_t is newly compromised hosts; D_t counts decoy diversions; A_t, F_t are newly reported true/false alerts; U_t is compromised but unisolated hosts; and C_T is critical compromised hosts still unisolated at the horizon:

$$\begin{aligned} r_t = & -0.16 I_{\text{response}} - 0.45 I_{\text{invalid}} \\ & + 1.50 I_{\text{true inspect}} - 0.18 I_{\text{clean inspect}} \\ & + 0.12 c_i I_{\text{patch}} \\ & + (1.10 + 0.55 c_i) I_{\text{true isolate}} \\ & - (0.65 + 0.85 c_i) I_{\text{false isolate}} \\ & + 0.08 v_i I_{\text{decoy}} - \sum_{i \in N_t} (2 + 4 c_i) + 0.75 D_t \\ & + 0.10 A_t - 0.06 F_t - 0.16 \sum_{i \in U_t} c_i \\ & - 1.4 \sum_{i \in C_T} 1. \end{aligned}$$

Action-specific terms use the acted-on host i. Coefficients are design weights, not calibrated monetary costs. Operational outcomes are reported separately from shaped return to reduce reward-hacking risk.

The CPU DQN uses a 58–128–128–25 MLP, replay buffer, target network, masked epsilon-greedy exploration, Adam, Huber loss and gradient clipping. Three training seeds (17, 29, 43) run for 280 episodes each. Baselines are Random Legal, Patch Priority, Alert Responder and Threshold Defense. Each baseline is tested on 30 held-out scenarios; each DQN checkpoint uses the same scenarios. DQN metrics are averaged across checkpoints within each scenario before summary statistics.

4. Results

Policy	Return	Critical	Uncontained
DQN	−7.34	0.644	1.244
Alert responder	−2.57	0.233	0.433
Threshold defense	−3.99	0.400	0.500
Patch priority	−13.87	0.933	2.100
Random legal	−15.58	0.667	1.933

Episode means are shown; critical assets have $c_i \geq 0.8$. DQN improves over random and patch-priority policies in return, but does not beat Alert Responder or Threshold Defense on return or containment. Mean detection delay is 0.961 steps for DQN, versus 1.469 and 1.593. No significance claim is made.

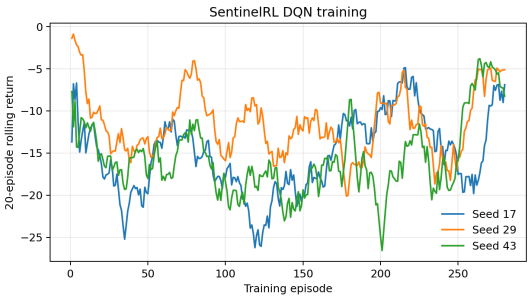


Fig. 1. 20-episode moving-average training return for the three seeds. All improve between first and last 20 episodes, but variance remains.

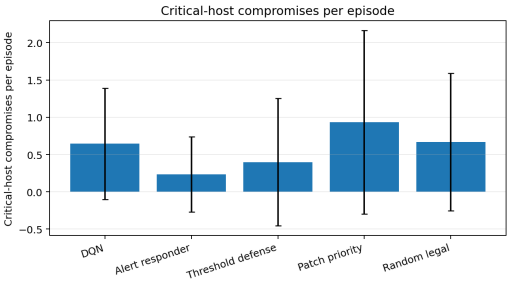


Fig. 2. Critical-host compromises on held-out scenarios; lower is better. Error bars show standard deviation.

5. Interpretation

DQN's mean critical-compromise count is 0.644 (scenario-level standard deviation 0.748); Alert Responder has 0.233 (0.504) and Threshold Defense 0.400 (0.855). DQN reduces mean detection delay in this run but leaves more active compromises by the episode horizon. The results show that a learned policy is not automatically superior to a carefully designed heuristic.

6. Attack-Rate Sensitivity

The trained agents were evaluated without retraining at attack probabilities 0.18, 0.23 and 0.28, with alert rates and budget fixed. Twenty scenarios were used per condition. At probability 0.23, mean critical compromises were 0.483 for DQN, 0.050 for Alert Responder and 0.250 for Threshold Defense. At 0.28 they were 0.567, 0.400 and 0.500 respectively. DQN is not best across all conditions; this sensitivity experiment is descriptive and small.

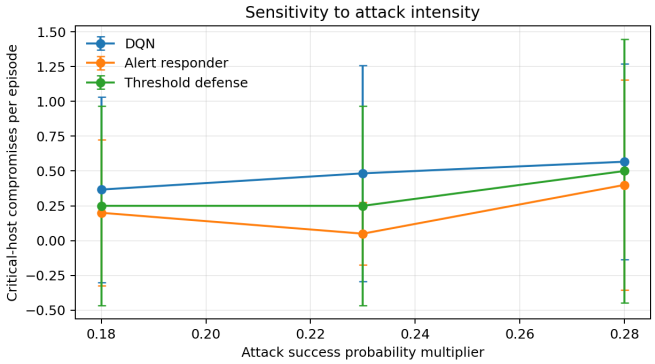


Fig. 3. Critical-host compromises as attack probability increases. DQN does not dominate the two strongest heuristics.

7. Limitations and Conclusion

The topology, attacker, alerts, criticalities and response budget are synthetic; the belief estimator is approximate; and the study has limited seeds and evaluation scenarios. We model no real packets, malware, operating systems, patch latency, alert correlation or adaptive human attacker. Results do not validate deployment. The supported conclusion is narrower: DQN learns a policy stronger than random and patch-only baselines, while transparent heuristic defenders remain better on aggregate return and containment. Future work should examine recurrent policies, richer telemetry and larger scenario sets.

Reproducibility and Safety

Code, checkpoints, tests, per-episode CSVs and plots are included. Run `python -m pip install -r requirements.txt`, `python -m pytest -q`, then `python run_all.py -episodes 280 -eval-episodes 30`. This CPU-only simulator does not connect to real systems and cannot validate operational cybersecurity.

References

1. Microsoft. *CyberBattleSim: An experimentation and research platform for automated agents in abstract simulated network environments*. 2021. https://github.com/microsoft/CyberBattleSim

2. "MininetGym: A modular SDN-based simulation environment for reinforcement learning in cybersecurity." *SoftwareX*, vol. 31, 102312, 2025. DOI: https://doi.org/10.1016/j.softx.2025.102312.

3. S. Vyas, V. Mavroudis, and P. Burnap. "Towards the Deployment of Realistic Autonomous Cyber Network Defence: A Systematic Review." *ACM Computing Surveys*, vol. 58, no. 1, 2025. DOI: https://doi.org/10.1145/3729213.

4. "Multi-Objective Reinforcement Learning for Automated Resilient Cyber Defence." *Applied AI Letters*, 2025. DOI: https://doi.org/10.1002/ail2.70007.